Week 2: Searching for Checkmate

Breadth-First and Depth-First Search

FIT1061: Introduction to Artificial Intelligence

2026-07-20

The Story: Deep Blue vs Kasparov

On May 11, 1997, something unprecedented happened. Garry Kasparov, widely regarded as the greatest chess player in history, resigned after just 19 moves in Game 6 of his match against a machine called Deep Blue. The machine won the match 3.5–2.5.

Deep Blue was built by IBM over more than a decade, at a cost of approximately $10 million. The project began as Feng-hsiung Hsu's PhD work at Carnegie Mellon University in 1985, originally called "ChipTest." By 1997, it had evolved into a supercomputer with 30 processors and 480 special-purpose chess chips, capable of evaluating roughly **200 million chess positions per second**.

Deep Blue didn't understand chess the way Kasparov did. It didn't have intuition, experience, or creativity. It *searched*. It explored millions of possible future positions, evaluated each one, and chose the move that led to the best outcome it could find.

After losing, Kasparov accused IBM of cheating: of having human grandmasters secretly guide the machine between games. In Game 2, Deep Blue had made an unexpected, almost creative move (move 36: Be4) that Kasparov couldn't explain. Years later, an IBM engineer suggested this move may have been the result of a software bug: Deep Blue played randomly when it couldn't decide between options.

IBM refused a rematch. They dismantled Deep Blue shortly after the match. The hardware was split between the Smithsonian Institution and the Computer History Museum. We still don't know the full story of what happened in those six games.

As Feng-hsiung Hsu later put it: *"The computer doesn't play chess like a human, any more than an airplane flies like a bird."*

The question Deep Blue raises is not whether machines can beat humans at chess; they clearly can. The question is **how**: what does it mean to search through millions of possibilities and find the right move?

That's what this week is about.

Game Trees

> **Further viewing**
>
> The 2003 documentary *Game Over: Kasparov and the Machine* (dir. Vikram Jayanti) tells the match from Kasparov's side: the cheating accusation, the mysterious Game 2 move, and the psychology of playing a machine. Watch on YouTube (85 min).

A simpler game first: tic-tac-toe

Before tackling chess, consider a simpler game. In tic-tac-toe, X goes first and has 9 possible opening moves. For each of those, O has 8 possible responses. For each of those, X has 7 options. And so on.

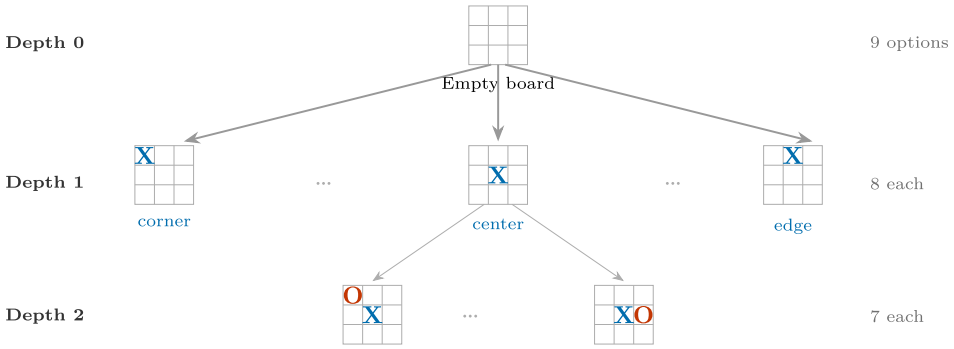


Figure 1: Tic-tac-toe opening tree: the first two levels of moves.

This creates a **tree**: a branching structure where each node is a game state and each edge is a move. The root is the starting position, and the leaves are finished games (someone wins, or it's a draw).

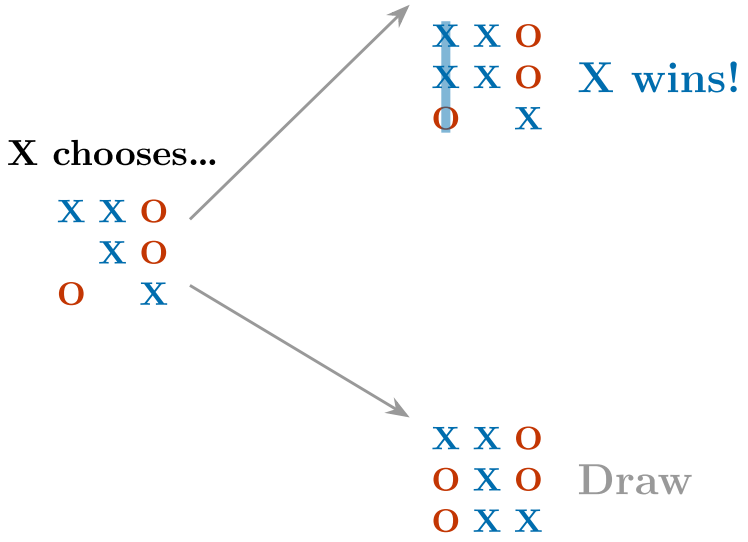


Figure 2: How a game ends: one decision point, two possible outcomes.

A **leaf** is a game state where no more moves can be made: either someone has won, or the board is full. Search means exploring branches of this tree to find a path that leads to a winning leaf.

How fast does the tree grow?

Even for tic-tac-toe, the tree grows quickly:

Depth	Positions	Scale
1	9	A small group
2	$9 \times 8 = 72$	A classroom
4	$9 \times 8 \times 7 \times 6 = 3{,}024$	A lecture theatre
9 (full game)	$9! = 362{,}880$	A small suburb

And tic-tac-toe is a *tiny* game.

Chess as a tree

Chess works exactly the same way, but the branching factor is roughly **35** legal moves per position instead of 9.

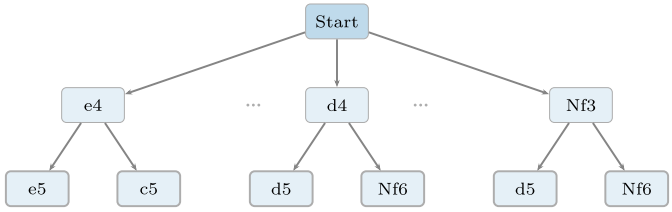


Figure 3: Chess game tree: only 3 of ~35 branches shown at each level.

The consequences of a branching factor of 35 are dramatic:

Depth	Positions	Scale
1 move	35	A classroom
2 moves	$35^2 = 1{,}225$	A lecture hall
4 moves	$35^4 \approx 1.5$ million	Population of Adelaide
10 moves	$35^{10} \approx 2.7 \times 10^{15}$	About 7 times the grains of sand on Bondi Beach

A typical chess game runs 40 moves deep. The full tree is unimaginably large. We cannot check everything. We need a *strategy* for deciding what to explore first.

Search Problems

Before choosing a strategy, we need a vocabulary for talking about search. Every search problem has four components:

Problem Formulation

1. **States**: configurations of the world (e.g., board positions, grid cells)
2. **Actions**: moves that transition from one state to another (e.g., legal chess moves, steps on a grid)
3. **Goal test**: a way to check whether we've reached the destination (e.g., checkmate, reaching a target cell)
4. **Path cost**: how expensive the path was (e.g., number of moves taken)

This formulation applies to every search algorithm you'll encounter this semester: BFS, greedy best-first, hill climbing, and beyond. The algorithms differ in their *strategy*, not their problem structure.

For the rest of this week, we'll search on a **grid**: a graph where each cell connects to its four neighbours (up, down, left, right). Some cells are walls (impassable). The goal is to find a path from a start cell to a goal cell.

Breadth-First Search (BFS)

The queue

BFS uses a **queue** as its frontier: the collection of nodes waiting to be explored. A queue is **first-in, first-out (FIFO)**: the first item added is the first item removed.

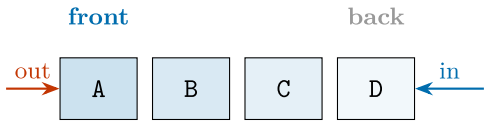


Figure 4: A queue: items enter at the back and leave from the front.

Think of a line at a coffee shop: the first person in line gets served first. This ordering is what gives BFS its key property.

The algorithm

Definition: Breadth-First Search

Explore all nodes at depth d before any node at depth $d + 1$. The queue ensures this: nodes discovered first are explored first.

BFS works as follows:

1. Start with the start node in the queue.
2. Remove the front node from the queue (dequeue).
3. If it's the goal, we're done: reconstruct the path.
4. Otherwise, add all unvisited neighbours to the back of the queue.
5. Repeat until the goal is found or the queue is empty.

Pseudocode

```
function BFS(start, goal):
    frontier ← Queue()
    frontier.enqueue(start)
    came_from ← {start: nil}
    nodes_expanded ← 0

    while frontier is not empty:
        current ← frontier.dequeue()
        nodes_expanded ← nodes_expanded + 1
        if current = goal:
            return reconstruct_path(came_from)

        for each neighbour of current:
            if neighbour not in came_from:
                came_from[neighbour] ← current
                frontier.enqueue(neighbour)

    return failure
```

Track `nodes_expanded` so you can report it later when comparing BFS to DFS. The DFS variant (next section) does the same.

The `came_from` dictionary

The `came_from` dictionary is the most important data structure in BFS. It does two jobs simultaneously:

1. Visited tracking. If a node appears as a key in `came_from`, we've already seen it. We don't add it to the queue again. Without this check, BFS would loop forever on graphs with cycles, rediscovering the same nodes over and over.

2. Path reconstruction. Each entry `came_from[B] = A` records that we reached node B from node A. To find the path from start to goal, we follow these links backwards: start from the goal, look up where we came from, repeat until we reach the start, then reverse the sequence.

Why BFS guarantees the shortest path

BFS processes nodes in **order of their distance from the start**. All nodes at distance 1 are explored before any node at distance 2. All at distance 2 before distance 3. And so on.

This means the *first* time BFS reaches any node, it must be via the shortest path, because every shorter path has already been checked.

> **!** Key Takeaway
>
> BFS guarantees the shortest path in unweighted graphs (where every step costs the same). This guarantee comes from two things: the **queue** (FIFO order) and the **uniform step cost**.

This guarantee breaks when steps have different costs: for example, roads with different travel times. That's a problem for later weeks.

Depth-First Search (DFS)

What if we swapped the queue for a stack? The shape of the algorithm stays identical; only one decision changes: which node to expand next.

The stack

A **stack** is **last-in, first-out (LIFO)**: the most recently added item is the first one removed. Think of a stack of plates: you take from the top, where you just put one.

The algorithm: one data-structure swap

DFS is BFS with `Queue` replaced by `Stack`, `enqueue` by `push`, and `dequeue` by `pop`. Everything else is identical.

```
function DFS(start, goal):
    frontier ← Stack()
    frontier.push(start)
    came_from ← {start: nil}
    nodes_expanded ← 0

    while frontier is not empty:
        current ← frontier.pop()
        nodes_expanded ← nodes_expanded + 1
        if current = goal:
            return reconstruct_path(came_from)

        for each neighbour of current:
```

```
            if neighbour not in came_from:
                came_from[neighbour] ← current
                frontier.push(neighbour)

    return failure
```

A different shape of exploration

Because the stack returns the most recently added node, DFS commits to one branch and follows it as deep as possible before backtracking. BFS radiates outward in layers; DFS snakes along a corridor until it hits a wall or the goal.

> **!** Key Takeaway
>
> DFS finds *a* path, not necessarily the shortest. Switching from FIFO to LIFO breaks the layer-by-layer ordering that gave BFS its shortest-path guarantee.

Why care about DFS, then? It is the basis of many other algorithms (backtracking search, topological sort, cycle detection) and the LIFO order is sometimes the right fit for the problem. In the lab you'll measure the tradeoff directly on a grid designed to expose the contrast.

Worked Example

Consider a 5×5 grid with some walls. Start at S = (0,0), goal at G = (4,4).

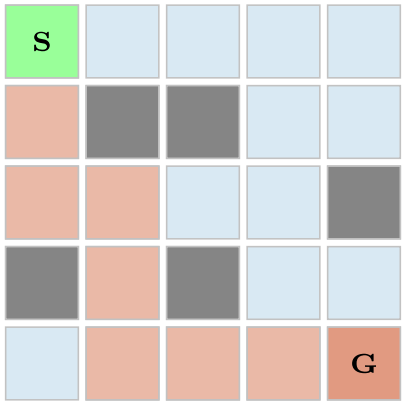


Figure 5: BFS result on 5×5 grid: blue = explored, orange = path.

BFS explores the grid **layer by layer**, radiating outward from the start like ripples in a pond:

- **Step 1:** Dequeue S. Add neighbours (0,1) and (1,0) to the queue.
- **Step 2:** Dequeue (0,1), the first item added. Explore its neighbours.
- **Step 3:** Dequeue (1,0), the second item added. Explore its neighbours.
- The pattern continues, expanding outward one layer at a time.

The pattern: BFS explores almost every open cell in the grid before the goal turns up. It guarantees the shortest path, but it makes you pay for that guarantee by exploring widely; and the larger the grid, the bigger that explored fraction grows.

In the lab you'll run BFS on this small grid and on a larger one yourself, recording the two numbers that matter (**path length** and **nodes expanded**) and compare them across grid sizes. Those counts are exactly what the exercise asks you to produce, so we leave them for you to measure.

BFS vs DFS on a dead-end grid

To see the BFS/DFS contrast clearly, run both algorithms on a grid containing a long dead-end corridor that leads away from the goal. You'll observe a pattern like this:

Metric	BFS	DFS
Path length	shorter (optimal)	often longer
Exploration shape	radial, layer-by-layer	snaking, committed to one branch
Nodes expanded	depends on grid	sometimes fewer, sometimes more
Shortest-path guarantee	yes (unweighted)	no

The lesson: one decision about the frontier flips the algorithm's behaviour.

Can we do better than blind exploration? What if the algorithm had a *sense of direction*, some way to guess which cells are closer to the goal? That's next week: **greedy best-first search**.

Summary

Key vocabulary:

Term	Meaning
State	A configuration of the world (e.g., a grid position)
Action	A transition from one state to another
Goal test	Checks whether we've reached the destination
Branching factor	Number of possible actions from each state
Queue (FIFO)	Data structure: first in, first out
Stack (LIFO)	Data structure: last in, first out
Frontier	The set of nodes waiting to be explored
`came_from`	Dictionary tracking visited nodes and path ancestry

Key takeaways:

- A **game tree** represents all possible sequences of moves. Even simple games have enormous trees.
- **BFS** explores layer by layer, guaranteeing the shortest path in unweighted graphs.
- **DFS** swaps the queue for a stack: same algorithm shape, different exploration order, no shortest-path guarantee.
- The **frontier data structure** determines the search behaviour: FIFO gives BFS, LIFO gives DFS.
- The `came_from` **dictionary** tracks both visited status and path reconstruction.
- BFS is **optimal** (shortest path) but **expensive** (explores broadly); DFS finds *a* path and explores differently.

What's next: BFS and DFS explore blindly. Next week, we give the search a *sense of direction*: greedy best-first search uses a heuristic to guide exploration toward the goal. It's faster, but there's a catch.

In the Lab

This week's task: BFS & DFS

1. **Hand-trace** BFS on a small personalised graph: frontier, visited set, exploration order at every step. Before any code.
2. **Implement** `bfs_search` in the provided notebook using the supplied `Queue`, `GridWorld`, and `reconstruct_path` helpers. Test on 5×5 and 10×10 grids.
3. **Adapt to DFS** in the same notebook by swapping `Queue` for `Stack` (and `enqueue`/`dequeue` for `push`/`pop`).
4. **Compare** BFS and DFS on the provided dead-end grid: path length, nodes expanded, cells explored.
5. **Reflect** on Deep Blue in a 150–250 word response: *When an AI wins, does it matter how?*